NAME: HARI VISHNU N
REG. NO: 192524319
COURSE CODE: CSA1023
COURSE NAME: SOFTWARE ENGINEERING

EXP 15:

Demonstrate how to work with Git branches and resolve merge conflicts when collaborating with others.

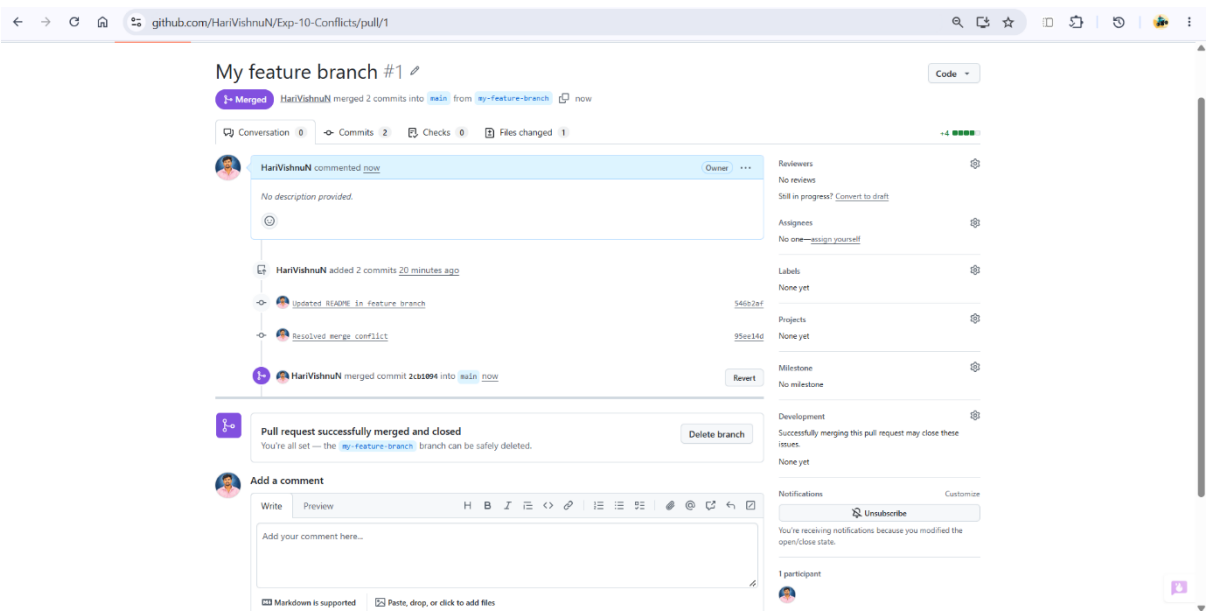
Aim:

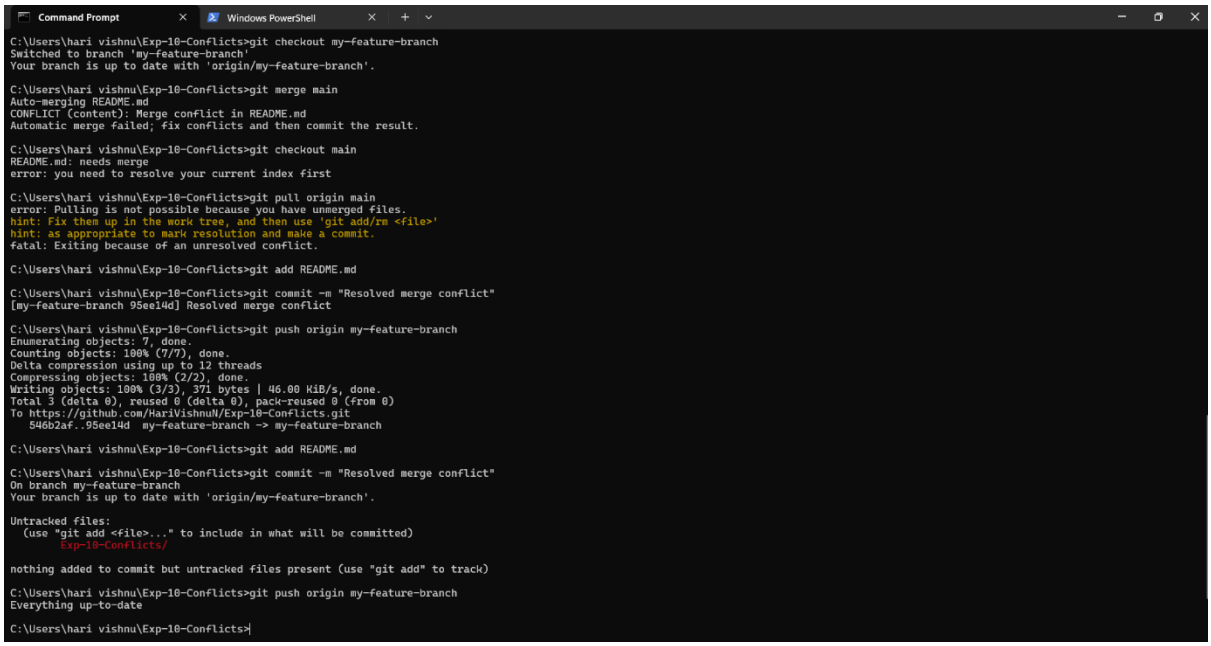
To demonstrate how to work with Git branches and resolve merge conflicts when collaborating with others..

procedure:

1. Clone a shared remote repository to the local workspace using git clone.
2. Create and switch to a new isolated feature branch using git checkout -b <branch-name>.
3. Modify a target file (such as README.md) in the local code editor and save the changes.
4. Stage the modified file with git add and save it to the branch history using git commit.
5. Upload the feature branch to the remote GitHub repository using git push.
6. Switch to the local main branch and use git pull to fetch concurrent remote updates.
7. Switch back to the feature branch and use git merge main to intentionally trigger a merge conflict.
8. Open the conflicting file in an editor, delete the Git conflict markers (<<<<<<<, =======, >>>>>>>), and manually combine the text.
9. Stage the resolved file with git add and commit the final merge to finalize the conflict resolution.
10. Push the updated feature branch to GitHub and open a Pull Request to propose merging the resolved code.

Outputs:





Summary of Tasks Performed: I cloned a shared repository and created a feature branch (my-feature-branch). I made local modifications to README.md, committed, and pushed them. To simulate collaboration, the main branch was updated concurrently on GitHub. Before merging, I switched to main and pulled the latest remote changes.
Conflicts Encountered: When switching back to my feature branch and executing git merge main, a merge conflict occurred in README.md because both branches had modified the exact same line.
Resolution Process: I opened the conflicting file in VS Code and located the conflict markers. I manually reviewed both versions, deleted the Git markers, and edited the text into a single combined sentence. I saved the file, ran git add to mark the conflict as resolved, committed the final merge, pushed the branch to GitHub, and created a Pull Request.

Result: The process of working with Git branches and resolving merge conflicts was successfully demonstrated. A feature branch was created, a merge conflict was intentionally triggered by simulating concurrent updates, and the conflict was manually resolved, committed, and successfully merged via a Pull Request.